

모바일 트랙 — 파운데이션·신뢰성 → 스포츠북 모바일 캡스톤

이 트랙은 무엇인가. Android(Kotlin + Jetpack Compose)로 모바일 앱 기본기를 쌓고, 신뢰성 (구조적 동시성·오프라인 정합·토큰 refresh 레이스·WebSocket 재연결)을 손으로 판 뒤, 네가 만든 sportsbook 백엔드를 소비하는 네이티브 베틱 앱 <code>sportsbook/mobile-client</code>로 달는다. "해의 베틱: 백엔드 + 게이트웨이 + Android"가 하나의 고용주 서사로 묶이는, 백엔드 트랙의 연장(enrichment)이다. 언제 시작하나. 이견 확장 트랙이다 — 백엔드 트랙을 끝낸 뒤 들어온다. 이유: ① Kotlin은 JVM 언어라 직전에 완주한 Java 21·Spring·Gradle Kotlin DSL·JUnit 자산이 그대로 전이되고, ② 캡스톤 <code>mobile-client</code>가 네가 만든 sportsbook 백엔드를 그대로 소비한다(같은 도메인 모델을 Kotlin DTO로 재기술). 읽는 법. 위에서 아래로 한 프로젝트씩. 각 블록은 자족적이다 — 노트 순서·재구성/커밋 활용·L3·검증·합정·완료 바가 블록 안에 다 있다. 이 문서 하나로 트랙을 완주하도록 썼다. 딱 한 번만 외우는 공통 어휘 (이후 각 블록은 이 말만 쓴다): - L3 = 구조적 동시성·코루틴 취소·StateFlow 소유권·오프라인 정합·토큰 refresh 레이스·WebSocket 재연결 등 <i>폭발 변경 큰 결정</i>. 답지 덮고 백지에서 코드+이유를 재구성하고 실패 모드까지 방어돼야 "끝". - L2 = 구조·흐름·책임 분리(펼쳐 설명 되면 충분). L1 = 보일러플레이트·스타일·설정(읽고 통과). - 핵심 동작 = <code>docs/commits/NNN.md</code> (답지)를 덮고, <code>docs/practice/NNN.md</code> (문제지)의 계약만 보고 직접 구현 → 답지와 diff. 답지 끝 <code>## 기억/설명 Level</code>에 L3/L2/L1이 적혀 있으니 <i>들어가기 전에 본다</i>. - 커밋 규율 (학습 산출물을 커밋할 때 — 이 트랙 전 레포 공통): 제목은 영어 명령형 <code><type>(<scope>): ...</code> (type ∈ feat·fix·docs·test·refactor·chore·build·ci·perf), 본문은 한국어 <code>【근거】</code> 왜 / <code>【변경】</code> 무엇 / <code>【검증】</code> 어떻게 (잡일·merge는 면제), AI 공동저자 트레일러 없음. 날짜는 KST·author==committer·근무시간 09:00-21:59·분초 랜덤·중복 금지. dual-form 문서는 답지 <code>docs(commits)</code> / 문제지 <code>docs(practice)</code> 스코프로 나눠 바뀐 파일만 add(−A 금지). 후속 fix는 새 phase로 답지 append. - △ 환경 한계(ENV-LIMIT) — 트랙 공통. JVM 단위 테스트(JUnit/MockK/Robolectric/Turbine/ MockWebServer)는 Android SDK·에뮬레이터 없이 돈다 → 여기가 "실제 실행" 게이트다. 빌드 (<code>assembleDebug</code>)와 Compose/Espresso 계속 테스트는 SDK·에뮬레이터가 필요하다 → SDK 머신에서 따로 검증한다. SDK 없는 환경에선 코드 리딩 + JVM 테스트로 돌고, "통과"를 허위로 적지 않는다.
--

전체 경로:

<code>mobile-foundations-training</code> → <code>mobile-reliability-training</code> → <code>*sportsbook/mobile-client</code> (캡스톤) (④·2B·입문 대장) (①+③·2A·대장 ★) (③+④·2A·백엔드 소비)
이 트랙 내부는 순차다. 캡스톤 <code>mobile-client</code> 는 모바일 페어(foundations+reliability) + sportsbook 백엔드가 전부 완료된 뒤 진입한다(자세한 의존은 맨 아래 "병렬성").

1. mobile-foundations-training — 입문 대장(Postboard 워밍업)

무엇/왜. Kotlin + Android SDK + Jetpack Compose 스파인을 한 앱(Postboard)으로 관통하며 "정상적인 앱 빌드 기본기"를 채운다 — 상태 호이스팅·내비·Room·Retrofit·Hilt·수동 페이지까지. 신뢰성 본대(2번)의 곡선을 부드럽게 하는 워밍업이다. 이 트랙에서 읽는 것. Compose 상태 소유권과 coroutine 생명 주기의 첫 감각, 단일 진실원천(Room) 습관.

장르·리듬. 장르 ④ 따라만들기(스파인 순차 구현) · 순차(2B) · merge 커밋 없음. 답지 17 / 문제지 17(1:1).

노트 읽는 순서 (<code>mobile-foundations-training/notes/</code> , 묶음별 — DESIGN 명시 순서). - 언어: <code>kotlin</code> - 플랫폼: <code>android-sdk</code> → <code>jetpack-compose</code> → <code>compose-navigation</code> → <code>viewmodel-stateflow</code> - 비동기: <code>kotlin-coroutines</code> - 연동: <code>retrofit-okhttp</code> → <code>room</code> - DI: <code>hilt</code> - 스타일: <code>material3</code> - 테스트: <code>junit-mockk</code> → <code>compose-ui-test</code> · 빌드: <code>gradle-android-kts</code>

처음 보는 스택(Compose·Room)은 1부 개념부터 깊게, 짝 `reference-impl` / 을 한 번 만진다.

재구성·커밋 활용법. 묶음으로 본다 — - 001~002 빌드 골격·Compose 상태/이벤트 — L3: 002 상태 호이스팅(상태 소유 vs 무상태 분리·recomposition). - 003~006 도메인·상태·내비·로컬 저장 경계 — L3: 004 sealed UiState + StateFlow 소유권, 005 라우트/ 백스택(SavedStateHandle), 006 Entity vs 도메인 분리·DAO Flow 단일 진실원천. - 007~010 HTTP/DI/동기화/CRUD — L3: 009 오프라인 우선(캐시 즉시 표시 + 백그라운드 갱신·combine 우선순위), 010 낙관적 삭제 + 롤백. - 011~015 상태 표시·폼·세션·페이징·이미지 — L3: 014 수동 페이징(page/limit·근접 로드·골/중복 가드). - 016~017 폭발 변경 큰 로직 + 화면 테스트.

L3 — 손으로 백지 재구성할 것. Compose 상태 호이스팅/recomposition(002), coroutine 생명주기 (viewModelScope , 005·006), Room 스קי마와 단일 진실원천(006), 낙관적 업데이트 & 롤백(010).

검증. `make test` (`testDebugUnitTest` — JVM 단위, 디바이스 불필요) / `make connected-test` (계속 — 에뮬레이터 필요·ENV-LIMIT).

합정. 여기는 *기초* 스코프다 — Coroutines/Room/Hilt 심화, WorkManager·Paging 3, DataStore/Keystore 토큰 보안, WebSocket/STOMP·JWT refresh·RFC 7807은 전부 신뢰성 레포 몫. 여기서 욕심내지 마라. SDK 없으면 빌드 불가 — 코드 리딩 + JVM 테스트로 돈다.

완료 바 → 다음으로. 상태 호이스팅·StateFlow 소유권·Room 단일 진실원천을 백지로 설명되면 — 이제 신뢰성 본대로. foundations는 워밍업이라 깊이를 욕심내지 말고, 진짜 L3는 다음 레포에 몰려 있다.

커밋 메모. 상단 <커밋 규율>대로(영어 명령형 제목 + `【근거】` / `【변경】` / `【검증】`, AI 트레일러 없음, KST 근무시간). dual-form 스코프: 답지 `docs(commits)` / 문제지 `docs(practice)` 분리, 변경 파일만 add.

2. ★ mobile-reliability-training — 모바일 대장(신뢰성 결정 패턴)

무엇/왜. 모바일 모듈의 대장(가장 두꺼운 통합, ★심화 봉우리). 구조적 동시성·오프라인 캐시·WebSocket/STOMP·토큰 refresh·성능·신뢰성 테스트를 4트랙(essential / advanced / sre / domain)으로 깊게 판다. frontend-reliability(114커밋)의 모바일 대응으로 기획됐다. 이 트랙에서 읽는 것. "비동기·오프라인·실시간이 <i>틀리게</i> 동작하는 모든 경우"를 코드로 막는 판단력 — 캡스톤 mobile-client에서 그대로 재사용할 L3.
--

장르·리듬. 장르 ①(코드 재구성) + ③(작업규율) · triplet(2A): `docs: define` → `feat` → `test` → `route` 연결 → 복잡도 회고 → merge. 답지 81 / 문제지 34 — 대형. 트랙별로 L3는 몇 개씩만. 결번(merge)이 정의돼 있으니 번호는 `docs/commits/README.md`로 확인.

노트 읽는 순서 (<code>mobile-reliability-training/notes/</code> , 묶음별 — DESIGN 명시 순서). - 동시성: <code>kotlin-flow</code> → <code>structured-concurrency</code> - 영속/오프라인: <code>room-offline-first</code> → <code>workmanager</code> → <code>paging3</code> → <code>datastore-security</code> - 네트워크/실시간: <code>okhttp-websocket-stomp</code> → <code>retry-backoff</code> → <code>jwt-auth-interceptor</code> → <code>problem-detail-mapping</code> - 성능: <code>compose-performance</code> - 테스트: <code>turbine</code> → <code>mockwebserver</code> → <code>espresso</code> → <code>screenshot-testing</code> → <code>hilt-testing</code>

(Kotlin·Compose 기초는 foundations 노트의 영역 — 여기서 재기술하지 않는다.)

재구성·커밋 활용법 — 4트랙으로 본다. - 저장소 기반 [000~003] — Gradle/버전 카탈로그·core UiState /dispatchers·Compose 호스트. - essential [005~033] — 곱 검증(005~009) · async 상태 Flow→StateFlow(011~015) · 여러 경계(017~021) · 페이징(023~027) · 낙관적 업데이트(029~033). L3: 012 flatMapLatest 취소, 018 상태 vs 일회성 이벤트. - advanced [035~058] — 오프라인 캐시(035~040) · WorkManager(042~046) · 실시간 STOMP(048~052) · 디자인시스템 신뢰성(054~058). L3: 036~038 DB 단일 진실원·write-through·실패 시 보존, 049~050 STOMP-dedup. - sre/resilience [060~083] — 카오스·재시도(060~064) · 토큰 refresh 레이스(066~071) · 재연결 스톱 (073~077) · 느린 API 경계(079~083). L3: 061 retryWithBackoff , 067·069 single-flight refresh (동시 401→단일 refresh), 074~075 backoff·교차연결 dedup·storm cap, 080 withTimeout · CancellationException 비·삼킴. - domain [085~095] — 오프라인 전략 감사(085~089) · 대형 리스트 가상화·예산(091~095). L3: 085~087 전략 표.

각 L3 구간의 docs: contract 를 읽고 덮고 직접 구현 → JVM 테스트(Turbine/MockWebServer) green → 답지 diff.

L3 — 손으로 백지 재구성할 것. 구조적 동시성/코루틴 취소(012·080·061), StateFlow 상태 소유권 (단일 비행·WhileSubscribed·상태 vs 이벤트), 오프라인 캐시 정합성(036~038·085~087), 토큰 refresh/ 인종 레이스 single-flight(067·069), WebSocket 재연결 + 역등(049~050·074~075).

검증. `make test` (`testDebugUnitTest` — JVM 단위; Turbine/MockWebServer/Robolectric) / `make connected-test` (계속 — 에뮬레이터 필요·ENV-LIMIT).

합정. 계속·스크린샷 테스트의 상시 실행은 ENV-LIMIT(SDK 머신에서). 공유 스택(Kotlin·Compose 기초·Hilt 기초)은 foundations 노트로 — 여기서 다시 익히려 들지 마라. 살아있는 백엔드 통합(gateway STOMP·JWT refresh의 실제 연결)은 캡스톤 몫이다 — 여기서 MockWebServer로 박제. 81커밋 전부 같은 무게로 돌지 말고 트랙별 L3만 깊게.

완료 바 → 다음으로. 구조적 동시성·single-flight refresh·오프라인 정합·STOMP 재연결을 백지로 방어되면 — 이제 캡스톤으로. 여기서 MockWebServer로 박제한 패턴들이 캡스톤에선 실제 sportsbook 게이트웨이를 향한다.

3. ★ sportsbook/mobile-client — 캡스톤: 살아있는 백엔드를 소비하는 베틱 앱

무엇/왜. 네가 만든 sportsbook 백엔드(8/8 e2e green)를 소비하는 네이티브 Android 베틱 앱. 모바일 페어 노트를 종합 적용하고, 신규는 살아있는 분산 백엔드 통합 3축(STOMP·OAuth2 refresh·역등 베틱)만 심화한다. 도메인 모델(Money·Odds·BetSlip...)은 shared-protocol 을 Kotlin DTO로 재기술의 종이 아니라 재기술 — 클라는 JSON만 본다). sportsbook/mobile-client 는 sportsbook/ 폴더 안의 독립 레포다. 이 트랙에서 읽는 것. "백엔드+게이트웨이+Android"를 한 서사로 묶은 커리어 산출물.
--

진입 체크. 모바일 페어(foundations+reliability) 완료 + sportsbook 백엔드 완료(gateway·betting·wallet·odds·feed 포함, e2e green). 실행은 `cd ../orchestration && ./scripts/build-all.sh && docker compose up -d --wait` 로 백엔드를 띄운 상태에서.

장르·리듬. 장르 ③ 계산산(실 API 계약 주도) + ④ 따라만들기 · triplet(2A) · 단일 모듈 앱. 답지 14(dev 001~012 + phase 2) / 문제지 12.

신규 학습 — Tier-1 3노트 (`sportsbook/mobile-client/notes/`). 나머지는 종합 적용이고, *이 3개만* 신규다: 1. `oauth2-refresh-against-gateway` — gateway는 RS256 검증만(발급자 없음). 401→refresh→재시도, single-flight로 동시 401을 단일 refresh로. (← 들어가기 전 선행) 2. `stomp-over-spring-broker` — 핸드셰이크 엔드포인트 ≠ 구독 목적지, 재연결 backoff·지터. 3. `idempotent-bet-contract` — 한 논리적 베틱 = 키 1개, 재시도마다 재사용. ODDS_DRIFT →재가격+새 키.

재구성·커밋 활용법. dev 커밋을 따라간다 — 002(domain: Money/Odds/BetSlip Kotlin 재기술) → 003(core: RFC 7807 problem+json 매핑) → 004(auth: 토큰 refresh 레이스, L3) → 005(data: offline-first events Room SoT·cursor 중분, L3) → 006(odds rest+live 병합) → 007(data: Idempotency-Key·RFC7807 역등 베틱, L3) → 008(wallet) → 009(realtime: 후기 STOMP client·핸드셰이크≠목적지·재연결, L3) → 010(Hilt: 3-클라이언트 분리로 DI 사이클 차단) → 011(Compose 화면·낙관적+롤백·drift 재확인) → 012(test: L3 JVM 박제). 004·005·007·009가 심장 — 덮고 재구성 후 답지 diff.

L3 — 손으로 백지 재구성할 것. - 토큰 refresh 레이스 — single-flight(004): 동시 401 N건이 각자 refresh하면 refresh 토큰 N번 소비 → authenticate 전체를 한 락으로, 진입 시 *cached.access ≠ 내가 쓴 토큰*이면 refresh 없이 재시도. (테스트: 동시 401 5건 → issuer 요청 정확히 1회.) + DI 사이클 차단: refresh는 인터셉터 없는 bare @AuthClient 로. - 역등 베틱(007/011): 한 베틱 = 키 1개, 재시도마다 재사용. ODDS_DRIFT (409, 부작용 없음)→재가격+새 키. 키는 ViewModel pendingKey 로 한 번만 생성(호출마다 새 키 = 안티패턴). - RFC 7807 매핑(003/007): safeApiCall 이 IOException→ProblemDetailException , AppError 가 ODDS_DRIFT→OddsChanged · DUPLICATE_BET→DuplicateBet 로 분기. - STOMP 핸드셰이크 ≠ 구독 목적지(009): `/ws/v1/odds` · `/ws/v1/bets` 는 연결 URL, 구독은 `/topic/odds/{eventId}` · `/user/queue/bets` . {userId} 는 서버가 CONNECT JWT sub 로 해소. + 재연결 지터. - offline-first SoT(005): UI는 Room만 관찰(오프라인 즉시 렌더), 네트워크는 캐시만 채움 (refresh=clear+upsert 한 트랜잭션). odds 병합점 단일화(006): REST seed + STOMP overwrite를 한 StateFlow로.

소비할 API 계약 (gateway `/api/v1/*` , `/ws/v1/*`). GET `/events` · `/odds` (공개) · POST `/bets` (JWT + Idempotency-Key 필수, 409 ODDS_DRIFT / DUPLICATE_BET , RFC 7807) · GET `/bets` · `/wallet/balance` (JWT) · STOMP `/ws/v1/odds` → `/topic/odds/{eventId}` (공개) · STOMP `/ws/v1/bets` → `/user/queue/bets` (CONNECT JWT). 15분 TTL access + refresh 회전.

검증. `make check-docs` (SDK 불필요 — 항상 그린) / `make test` (JVM 단위 — MockWebServer/MockK로 L3 박제) / `make build` (`assembleDebug` · ENV-LIMIT) / `make connected-test` (계속·ENV-LIMIT). refresh 기계는 MockWebServer로 green, 실 로그인 e2e는 발급자 부재(게이트웨이는 검증만·발급자 없음)로 ENV-LIMIT — 허위 "통과" 주장 없음.

합정. 발급자 부재: gateway는 검증만, 시스템에 토큰 발급 서비스 없음(정직한 ENV-LIMIT). STOMP 혼동: `/ws/v1/odds` 를 구독하면 (브로커에 그 목적지 없어) 영구 무음. 역등 키 안티패턴: `place()` 마다 새 키 = 재시도다 새 베틱. 키는 ViewModel 상태에 한 번만.

완료 바 → 트랙 완주. 백엔드를 띄운 상태에서 single-flight refresh·역등 베틱·STOMP 구독을 백지로 방어 되고 JVM L3 박제가 green이면 완주(빌드·계속은 SDK 머신에서). GitHub push로 커리어 산출물로 남긴다. 마지막으로 면접 대비로 동시성 · 역등성 · 실시간(STOMP) · 인종·보안 · 클라이언트 신뢰성(스테일·낙관적·오프라인)을 백지 설명으로 달는다.

커밋 메모. sportsbook/mobile-client 는 독립 레포라 그 레포 세션이 마지막에 한 번 커밋·푸시한다. 상단 <커밋 규율>대로 phase 경계(구현 블록 → 메타·문서 블록) 유지, 변경 파일만 add. 후속 fix는 새 phase로 답지 append.

병렬성 — 이 트랙의 순서 규칙

- 트랙 내부는 순차(병렬 불가). 캡스톤 `mobile-client`는 모바일 페어 + sportsbook 백엔드가 전부 완료된 뒤 진입한다. 즉 신뢰성을 먼저 끝내야 캡스톤이고, 게다가 백엔드 트랙의 sportsbook까지 끝나 있어야 한다(캡스톤이 그 백엔드를 실제로 소비하므로). 백엔드와 달리 두 겹의 선형이 있는 셈.
- 트랙으로서의 위치: 모바일은 백엔드 트랙의 연장이다(백엔드 완주 → 모바일 페어 → mobile-client). Kotlin이 JVM 자산을 전이받고, 캡스톤이 sportsbook 백엔드를 재사용하기 때문. 그래서 42·프로토타와 별개로 백엔드를 끝낸 사람의 다음 수에 가깝다.
- 모바일 페어 자체의 선행: foundations→reliability는 순서 고정 사슬이다. 페어 진입의 하드 선행은 M0뿐이지만, 자산 시너지·캡스톤 의존 때문에 백엔드 트랙 뒤에 두는 것이 권장이다.